

완전탐색, 백트래킹, flood fill

👤 소유자	🐼 암암코딩
☰ 태그	
🕒 생성 일시	@2024년 8월 15일 오후 12:51

완전탐색이란?

모든 경우의 수를 탐색하는 방법이다.

1) 브루트포스 (brute force)

단순히 조건문/반복문등을 사용해 모든 경우의수를 계산하여 답을 구하는 방법, N이 커지는 사용할수 없다.

2) DFS/BFS

완전 탐색의 경우의 DFS/BFS.가 많이 사용된다. 가령 길찾기/ 최단거리 탐색/ 가능한 조합 중 최적의조합을 모든 경우를 탐색하여 찾아내는 경우가 있다.

완전탐색의 경우는 데이터의 개수(N)이 작거나, 또는 임의 답 하나를 선택하거나, 역추적이 가능한경우에 주로 사용된다.

백 트래킹이란?

답을 찾는 도중에 답이 될 수 없다고 판단하면, 되돌아가서 다시 찾아가는 기법

DFS 탐색을 하면 모든경로를 다 탐색한다.

모든 경로를 탐색하기 때문에 불필요한 것같은 경로를 사전에 차단하지 않는다. 그래서 경우 수가 줄어들지 않는다.

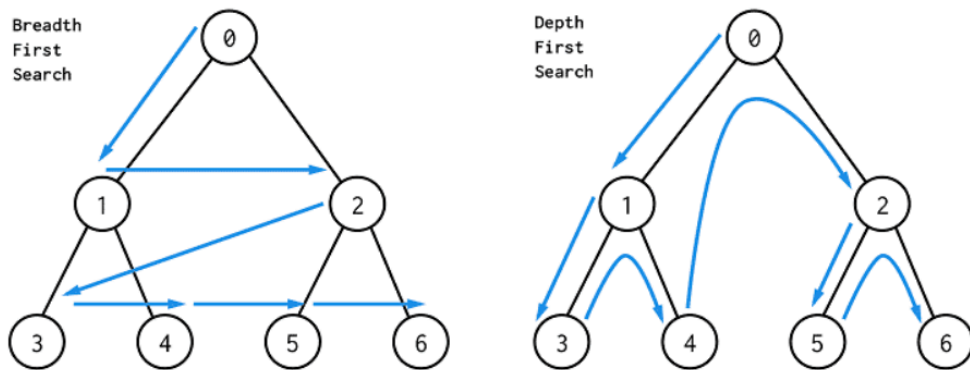
백트래킹은 완전탐색하는 도중에 탐색에 들어가지 않고 제한조건을 걸어서 특정 조건을 위 배가 되는지 안되는지 판단한후에 외배가 되는 경우는 제한하고 탐색한다.



다만 백트래킹 문제를 풀 때는 백트래킹인지 DFS 인지 분석해가면서 풀면 시간에 촉박하다. 일단 DFS 먼저 구현을 한 후에 제외시켜야할 조건이 있다면 추가시켜서 백트래킹 문제가 자연스럽게 구현이 되는 것이다.

백트래킹은 당연하게도 DFS/BFS 둘다 구현이 가능하다.

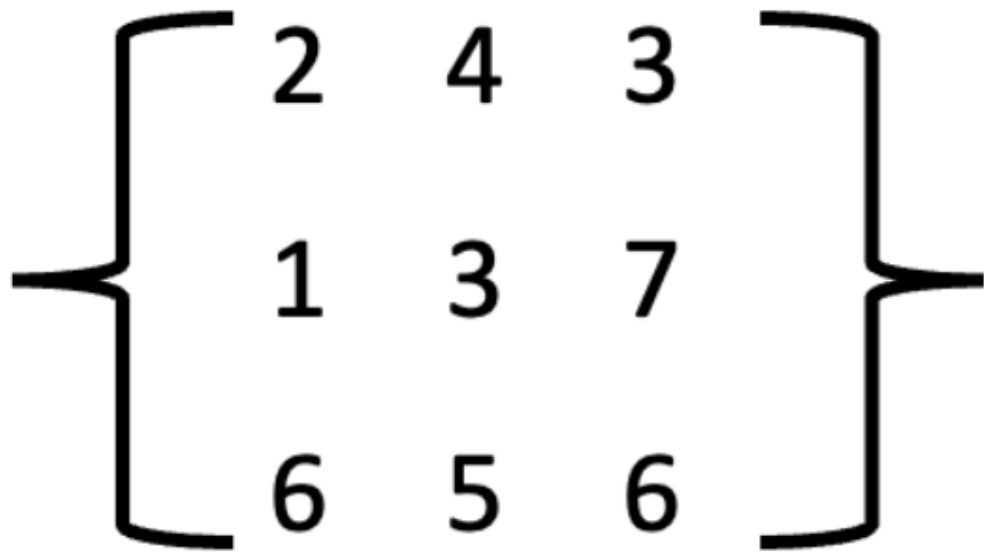
다만 이전 수행으로 돌아가야 하기 때문에 BFS보다는 DFS가 구현이 더편한경우가 많아서 주로 DFS를 이용해서 문제를 해결한다.



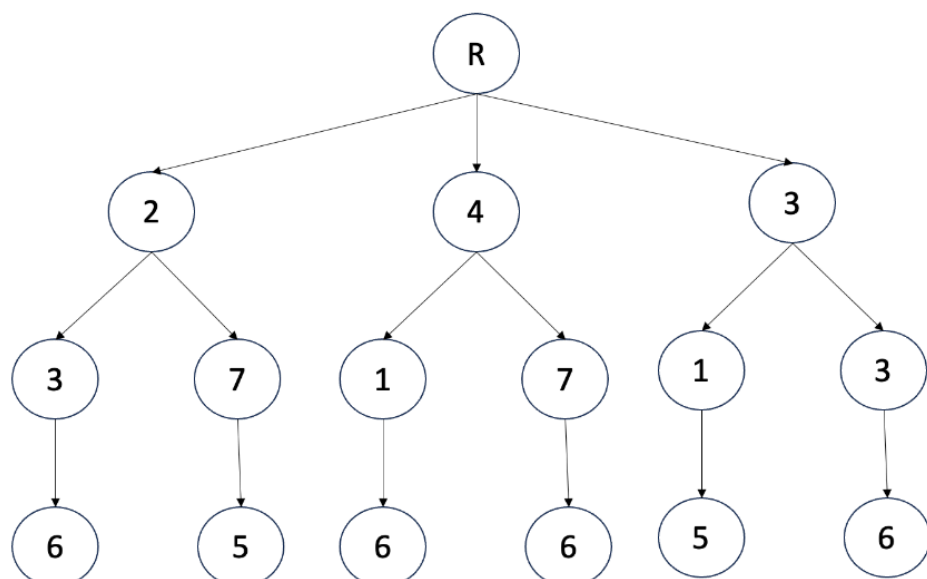
백트래킹 예시

3×3 행렬 선택 게임으로 예시를 들겠다.

규칙 : 아래와 같은 행렬이 존재 할때 3개의 수자를 선택하는데, 단 선택한 숫자들과 행과 열은 모두 중복이 되면 안된다.(즉 뽑아내는 숫자의 행과 열이 모두 달라야한다.)



트리로 표현해보면



```
#include <iostream>
```

```
int matrix[3][3] =
```

```
{
    {2,4,3},
    {1,3,7},
```

```

    {6,5,6}
};

bool check[3] = {};

void dfs(int row, int score)
{
    if (row == 3)
    {
        std::cout << score << std::endl;
        return;
    }

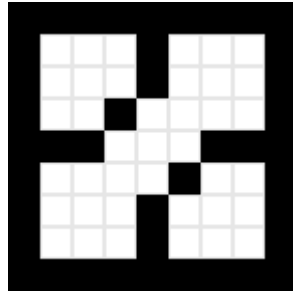
    for (int i = 0; i < 3; i++)
    {
        if (check[i] == false)
        {
            check[i] = true;
            dfs(row + 1, score + matrix[row][i]);
            check[i] = false;
        }
    }
}

int main()
{
    dfs(0, 0);

    return 0;
}

```

Flood Fill (플루드필)



주어진 시작점으로 부터 연결된 영역들을 찾는 알고리즘

다차원 배열의 어떤 칸과 연결된 영역을 찾는 알고리즘

예를 들어 그림판의 채우기 기능, 지뢰찾기에서 특정 셀을 클릭시 주변에 지뢰가 없는 모든 셀을 찾아주는 기능

```
#include <iostream>
```

```
//flood fill
```

```
char map[9][10] =
```

```
{  
    "#####",  
    "#...#...#",  
    "#...#...#",  
    "#..#...#",  
    "###...####",  
    "#...#..#",  
    "#...#...#",  
    "#...#...#",  
    "#####"  
};
```

```
void printMap()
```

```
{  
    for (int i = 0; i < 9; i++)  
    {  
        for (int j = 0; j < 10; j++)  
        {
```

```

        std::cout << map[i][j];
    }
    std::cout << std::endl;
}
}

void FloodFill(int x, int y)
{
    if (map[y][x] == '.')
    {
        map[y][x] = '@';

        FloodFill(x, y + 1);
        FloodFill(x - 1, y);
        FloodFill(x, y - 1);
        FloodFill(x + 1, y);
    }
}

int main()
{
    printMap();

    std::cout << "===== " << std::endl;
    FloodFill(4, 4);

    printMap();

    return 0;
}

```

연습문제 (앞 레벨과 중복되는 문제들이 있습니다. 해당문제는 STL을 사용하여 다른 방식으로 다시 풀어주세요.)





문제 2번

시작 좌표에서 종료 좌표까지 최소 몇 회만에 갈 수 있는지 확인해주세요.

BFS 알고리즘을 사용하여 길을 찾아주세요.

아래 1의 위치로는 이동할 수 없습니다.

2차원 배열의 맵은 하드코딩하여 풀어 주세요.

1	1		1
1		1	

[입력]

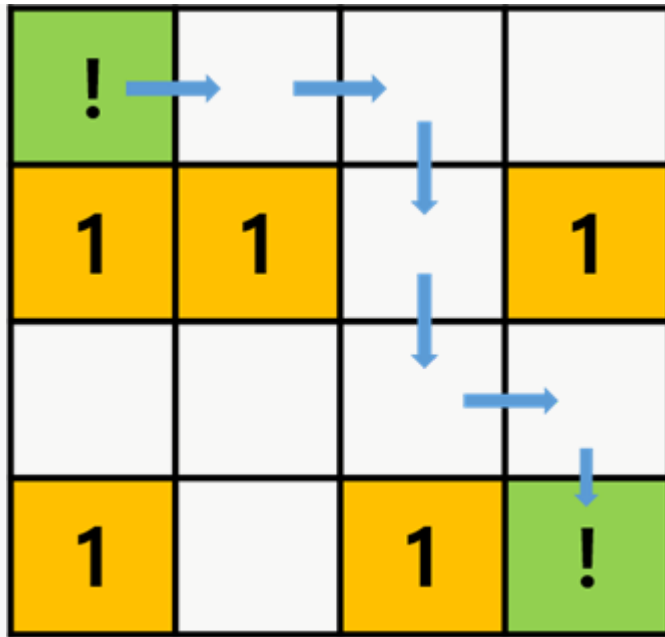
시작 좌표 Y X 와, 그 다음 줄에 종료 좌표 Y X 를 입력하세요.

[출력]

시작 좌표 부터 종료 좌표까지 최소 몇번 만에 도착할 수 있는지 출력해주세요.

[예시 1]

시작 좌표 (0, 0) 과 도착 좌표 (3, 3) 이 입력 될때, 결과는 6회입니다.



입력 예제

0 0

3 3

출력 결과

6회



(0, 0)에 서있는 쥐는 **치즈를 먼저 먹고**, 친구를 만나러 가려고 합니다.

쥐 			친구 	
				

쥐는 $\uparrow \downarrow \leftarrow \rightarrow$ 로 이동 할 수 있습니다.

[입력]

치즈 위치 (Y, X) 와 친구 좌표 (N, M) 을 입력 받으세요.

[출력]

친구를 만나기까지의 최단 거리를 출력 해주세요.

한 칸을 이동했을때가 1입니다.

[예시]

위 예제에서는 **9회** 이동으로 치즈를 먹고 친구를 만날 수 있습니다.

입력 예제

2 0

0 3

출력 결과

9



4 × 6 사이즈의 맵에 있는 모든 치킨을 먹으려고 합니다.

지혁이는 [0, 0]에서 출발하고, ↑ ↓ ← → 방향으로 이동할 수 있습니다.

0	0	1	2	1	2
1	0	0	1	0	1
0	0	1	2	0	0
2	0	0	0	0	0

1은 벽이라 이동할 수 없습니다.

2는 고기입니다.

0으로 표시된 지역만 이동할 수 있습니다.

[입력]

4 X 6 의 2차원 배열 정보를 입력 받으세요.

[출력]

치킨을 총 몇 마리 먹을 수 있는지 출력하세요.

입력 예제

0 0 1 2 1 2

1 0 0 1 0 1

0 0 1 2 0 0

2 0 0 0 0 0

출력 결과

2



현준이가 설치한 폭죽은 1초당 한번씩 불꽃이 사방으로 퍼져나갑니다.

폭죽이 한번 터질 때 마다 8방향 으로 터져 나가고, 연달아 8 방향
(←↖↗→↘↙↕↔)으로 또 터집니다.

만약 0초에 폭죽이 1개 있다면,

1초에는 폭죽이 8개 불꽃으로 터집니다.

2초에는 8개 불꽃이 각각 8개씩 64개로 퍼집니다.

3초에는 64개 불꽃은 $64 \times 8 = 512$ 개로 퍼집니다.

먼저 4 x 5 사이즈의 맵을 입력 받으세요.

0은 빈 하늘을, 1은 폭죽을 의미합니다.

[입력]

4 X 5 사이즈의 맵을 입력 하세요.

[출력]

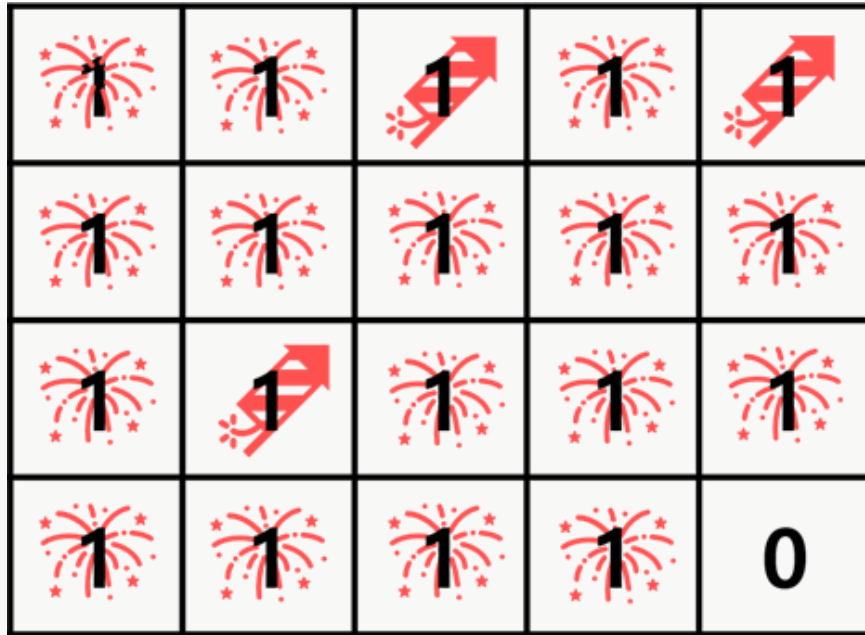
하늘 전체를 불꽃으로 채우는데 걸리는 시간을 출력 하주세요.

[예시 1]

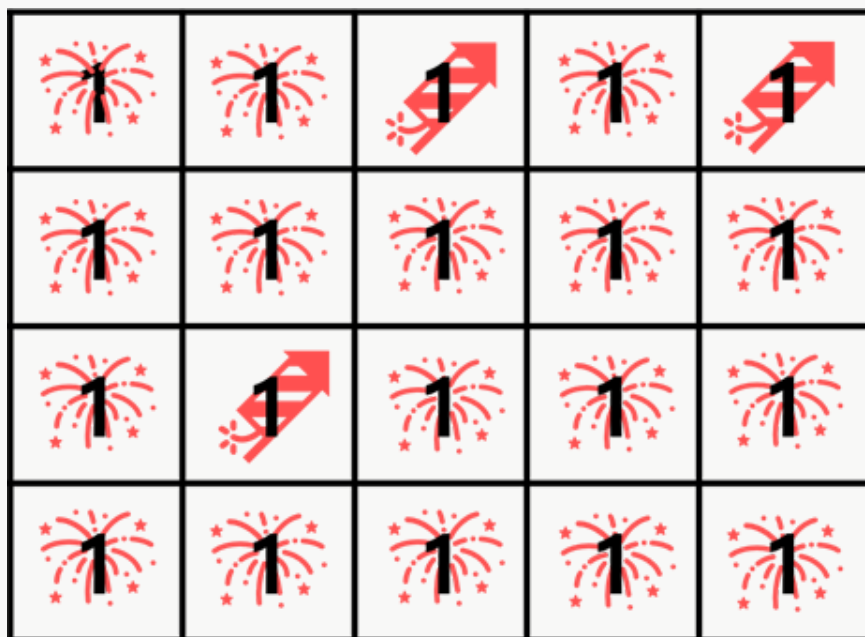
[1초 경과]

0				
				
			0	0
			0	0

[2초 경과]



[3초 경과]



예시 1에서 하늘을 불꽃으로 모두 채우는데 걸리는 시간은 3초 입니다.

입력 예제

0 0 1 0 1

0 0 0 0 0

0 1 0 0 0

0 0 0 0 0

출력 결과

3



지효는 땅이 얼마나 큰지 알아내는 드론을 띄웠습니다.

↑ ↓ ← → 으로 땅이 붙어있는 경우는 하나의 땅입니다.



[입력]

4×4 땅 상태를 입력 받아 주세요.

 1	 1	0	 1
0	 1	0	 1
0	 1	 1	0
 1	0	0	0

[출력]

(0, 0) 부터 상, 하, 좌, 우 네 방향으로 붙어있는 육지의 크기를 출력해주세요.

위 예제에서 사이즈는 5 입니다.

입력 예제

1 1 0 1

0 1 0 1

0 1 1 0

1 0 0 0

출력 결과

5



배달맨은 배달지역 가장 가까운 숫자를 향해 달려 갑니다.

1 ~ 4번 배달지역이 있고, 배달맨은 1 2 3 4번 지역을 한번씩만 들리면 됩니다.

(0, 0)에서 차량은 출발합니다.

만약 아래와 같이 입력받았다면, 아래 그림과 같습니다.

숫자는 지나갈 수 있는 지역, #은 벽입니다.

30002

##4##

01024



3				2
		4		
	1		2	4

배달맨은

(0,0)에서 가장 가까운 1번 지역에 도착합니다.

그리고 도착한 지역에서 가장 가까운 2번 지역에 도착합니다.

2번 지역에 도착하면, 가장 가까운 3번 지역에 도착합니다.

3번 지역에 도착하면, 가장 가까운 4번 지역에 도착합니다.

(거리가 같은 경우는 입력으로 주어지지 않습니다)

총 몇 번의 이동만에 모든 배달을 마칠 수 있을까요?

입력 예제

30002

##4##

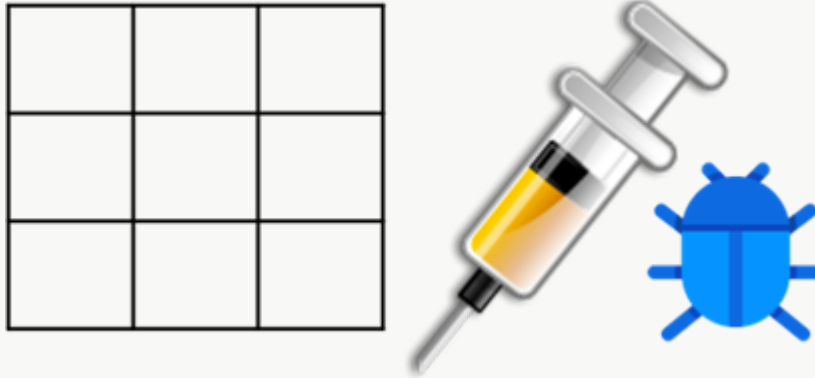
01024

출력 결과

15회



바이러스를 투여하고, 바이러스가 모두 퍼지고 난 뒤의 결과를 출력해주세요.
 바이러스는 두 곳에 투여되며, 상, 하, 좌, 우 네 방향으로 퍼지게 됩니다.
 바이러스가 퍼지는 맵의 크기는 3 X 3 입니다.



[입력]

바이러스가 투여되는 좌표 Y X 두곳을 입력하세요.

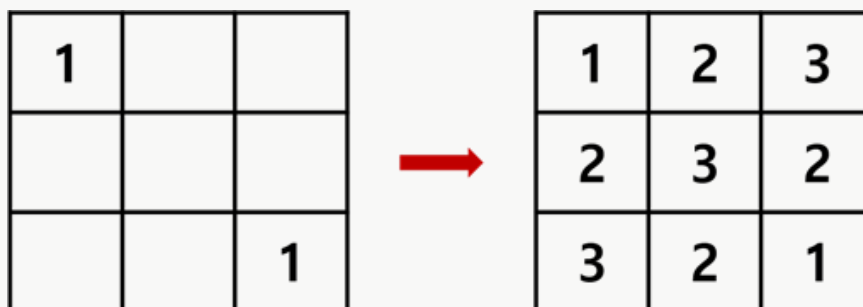
[출력]

바이러스가 모두 퍼지고 난 뒤의 상태를 출력해주세요.

[예시 1]

(0, 0) 과 (2, 2) 에 바이러스를 투여한다면 아래처럼 바이러스는 상하좌우 네 방향으로 퍼지며, 강도는 1씩 증가합니다.

(0, 0) 과 (2, 2) 에서 바이러스가 모두 퍼지면 위와 같은 결과가 출력되어야 합니다.



[예시 2]

만약 (2, 1) (2, 2)에 바이러스를 투여한다면 아래와 같은 결과가 출력되어야 합니다.

	1	1



4	3	3
3	2	2
2	1	1

입력 예제

0 0

2 2

출력 결과

123

232

321